Coordination Contracts: Trace-Derived Admission for Shared CPS State

Anonymous authors
Replace with author names and affiliations for camera-ready

Abstract

In cyber-physical systems (CPS), correct messaging does not imply correct coordination. A workflow may deliver every packet and still admit split-brain ownership conflicts, stale writes, reorder-sensitive violations, or out-of-scope updates to shared state. We present *Coordination Contracts*, a portable coordination-admission semantics for shared keyed state that combines keyed authority, temporal admissibility, reason-coded denials, and trace-derivability at the write boundary. The key design constraint is *trace-derivability*: verdicts must be reproducible from event traces plus declared configuration alone, without privileged hidden state. On the current benchmark artifact (54-sequence challenge corpus), the full contract matches the expected per-event verdict vector on all sequences and achieves event-level precision/recall/F1 of 1.0 (TP=37, FP=0, FN=0). Comparator decomposition shows that temporal-only proxies miss split-brain failures, while authority-only proxies miss stale and reorder-sensitive failures; the full contract improves over timestamp-only baselines by 13 denied violations. Validation overhead remains bounded in this trace-driven single-process setting, with event-level median/p95/p99 latency of 4.6/11.8/84.5 μs and bootstrap confidence intervals reported in the artifact. We also demonstrate transport-boundary semantic parity across event-log and LADS-shaped ingestion paths for canonical sequences. Optional async stress sweeps characterize sensitivity to delay, skew, and reorder; they support robustness discussion but do not replace the headline corpus verdict. This paper does not claim Byzantine tolerance, partition-safe consensus, or live cross-transport deployment equivalence.

1 Introduction

Communication is not coordination. In CPS orchestration, the correctness-critical question is not whether a message arrived, but whether a proposed write is admissible under explicit authority and temporal rules for shared keyed state.

Motivating vignette. Controller A releases a functional unit after finishing a task. Controller B, operating on stale assignment state, issues a new write on the same key. Transport delivery succeeds. Coordination still fails unless a shared-state admission layer denies the write. Coordination Contracts make this failure explicit at the write boundary with reason codes (e.g., `split_brain`, `stale_write`) before state mutation.

This paper proposes Coordination Contracts as a reusable coordination-admission layer above transport and below application policy. The model defines typed state, keyed authority, temporal admissibility, and transition validity. A pure validator consumes replay state and event fields and returns `allow`/`deny`with reason codes. This paper does not claim Byzantine tolerance, partition-safe consensus, or live cross-transport deployment equivalence.We instead claim something narrower and, we argue, practically valuable: a portable semantics layer that makes shared-state mutation auditable, replayable, and denyable under explicit authority and time rules.

Why now. Regulated CPS programs increasingly require post-incident traceability, evidence-preserving change control, and cross-team interoperability between heterogeneous controllers. A trace-derived admission layer is useful in this setting because it yields replayable enforcement decisions with explicit denial reasons, even when messaging succeeds.

The paper makes four claims:

1. A trace-derived validator can detect and deny the specified coordination failure classes on the benchmark corpus.
2. Validation is reproducible from traces plus declared configuration without privileged hidden state.
3. The model is transport-agnostic by construction at the event/state boundary; parity is shown for reference ingestion paths.
4. Per-write validation overhead is bounded on the evaluated workload.

2 Related work and positioning

Contract Net introduced higher-level coordination semantics for distributed task sharing [**?**]. TLA+ provides a strong foundation for formal specification of concurrent and distributed systems [**?**]. ROS 2 and DDS provide transport, discovery, and data-centric publish-subscribe semantics for robotic and distributed applications [**?**, **?**, **?**]. Log-based systems such as Kafka provide durable replay and keyed retention [**?**, **?**]. Classic optimistic concurrency and lease protocols address contention and stale ownership windows in distributed settings [**?**, **?**]. OPC UA LADS provides rich laboratory information and orchestration-facing device semantics [**?**, **?**].

Coordination Contracts are positioned differently: a portable write-validity layer for shared keyed state that combines explicit keyed authority, temporal admissibility, reason-coded denials, trace-derivability, and boundary-level transport portability. The goal is not to replace transport, industrial information models, or formal protocol methods. It is to make coordination admissibility auditable and replayable at the boundary where shared-state mutation is decided. In this sense, the paper argues for a missing systems layer: transport decides delivery, storage decides persistence, and coordination contracts decide whether a shared-state mutation is admissible.

This distinction is also what separates the present contribution from simpler reductions to timestamps, locks, or schema checks. Timestamp-only reasoning is insufficient for ownership conflicts; ownership-only reasoning is insufficient for stale or reorder-sensitive writes. Likewise, a device information model can describe state without deciding whether a candidate write is valid under explicit multi-writer authority and time rules. The paper's novelty is therefore in the combination, not in any one ingredient considered in isolation.

Positioning against adjacent mechanisms. Ordering policies, optimistic concurrency, leases, and locks address overlapping but distinct concerns: they govern *when* a write may proceed under a particular concurrency protocol, not necessarily whether the write is admissible under an explicit multi-writer authority model for shared keyed state. Consensus and distributed transactions provide stronger guarantees under assumptions (membership, failure models, transactional boundaries) that this work does not adopt. This is not baseline emulation; it is boundary ingredient isolation. Coordination Contracts provide a lightweight, trace-replayable admission predicate at the write boundary, complementary to those mechanisms.

3 Problem setting and failure model

We consider multiple writers updating shared keyed state in CPS.

Failure classes.

1. **Split-brain ownership conflict:** concurrent incompatible writers on one key.
2. **Stale write:** write based on outdated temporal state.
3. **Reorder-sensitive violation:** delayed/reordered writes violating declared acceptance order.
4. **Unknown-key access:** update outside declared contract scope.

In the evaluated artifact, temporal ordering is reflected in per-event timestamps and the validator's monotonicity rules over admitted prefixes; reordering is modeled by permuting delivery order in corpus sequences while holding the same event fields.

Assumptions. Identity fields in traces are treated as truthful in this artifact. Validation occurs before mutation. The evaluated harness is trace-driven and single-process. We make no live deployment guarantees under Byzantine actors, network partitions, or arbitrary runtime interleavings beyond those reflected in the benchmark and auxiliary stress artifacts.

4 Contract model and validator semantics

Definition 1 (Contract state)**.** *For each key k, contract state stores typed value v_k, owner o_k, temporal summary τ_k, and optional lease metadata ℓ_k.*

Definition 2 (Trace-derivability)**.** *A validator is trace-derived if verdicts depend only on replay state from admitted prefix, event fields, and declared configuration C.*

The validator interface is:

$$V_C(S, e) \to (\texttt{allow} \mid \texttt{deny}, R),$$

where R is a reason-code set.

Notation summary. S denotes replay state reconstructed from admitted events; e denotes the candidate event; C denotes declared contract configuration; R denotes reason-code output; o_k denotes owner for key k; and τ_k denotes per-key temporal summary used for monotonicity checks.

Proposition 1 (Determinism)**.** *For fixed C, S, and e, the validator returns a unique verdict and reason-code set.*

Proposition 2 (Admitted-prefix invariants, sketch)**.** *Let state evolve by applying only events with verdict `allow`. Then ownership consistency and per-key temporal monotonicity are preserved along admitted prefixes under the declared contract predicates.*

Scope note. The determinism property is aligned with the Lean formalization in `formal/lean/Labtrust/W2Contract` (validator interface and trace-shaped reasoning); admitted-prefix invariants are presented as a proof sketch here and are not claimed as a complete mechanized proof of live distributed execution.

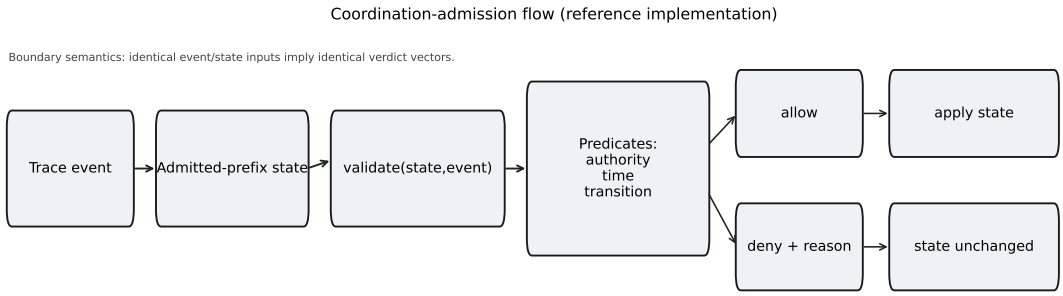

Figure 1: Coordination-admission flow. A candidate event passes through contract predicates and yields an `allow`/`deny`verdict with reason codes; state is updated only on `allow`.

Worked example. Suppose the admitted prefix already records owner `controller_a` for key `t1`. A candidate `task_start` from `controller_b` on `t1` yields verdict `deny` with reason `split_brain` because authority and ownership predicates fail. State is unchanged. If instead the candidate is a stale `task_end` whose timestamp violates monotonicity, the verdict is `deny` with `stale_write` (or related temporal reason codes in the artifact). This makes coordination failure explicit before application of the write.

5 Reference implementation and interoperability boundary

The reference system separates a pure validator from a contract-enforcing store. Writes are applied only on `allow`.

The transport claim is boundary scoped: if two ingestion paths induce identical event/state semantics at the contract boundary, they yield identical verdict vectors.

For OPC UA LADS-shaped interoperability, functional units or device-facing keys map to contract keys; controlling agents map to writers; LADS-compatible edges map to admissible transitions. The contract does not replace the LADS information model; it adds a portable admission check for writes alongside it.

6 Experimental methodology

6.1 Benchmark corpus and metrics

The current artifact evaluates a 54-sequence tiered corpus spanning positive controls, split-brain conflicts, stale writes, reorder-sensitive violations, unknown-key cases, and adversarial families such as cross-key interleaving, delayed release/reassignment, and concurrent controller races. Primary metric is exact per-event verdict-vector match per sequence. Secondary metrics include event-level TP/FP/FN, precision/recall/F1, class-level decomposition, event-level latency percentiles with bootstrap CIs, and boundary-level transport parity on canonical sequences.

6.2 Ground truth and labeling

Each sequence in `bench/contracts/corpus` carries an `expected_verdicts` vector defined independently of any particular validator implementation (manual design with review, or generation with fixed seeds where applicable; see the benchmark specification). The evaluation checks that the

Table 1: Corpus composition by coarse family (from frozen run; heuristic family mapping over sequence identifiers).

Family	Sequence count
control	14
split_brain	9
stale_write	8
reorder	7
unknown_key	1
other/adversarial/mixed	15

contract's per-event verdict vector matches this ground truth. The corpus fingerprint and script version in `eval.json` identify the exact snapshot used for the tables in this paper.

6.3 Failure-class inference and units of analysis

Class-level slices in this paper are derived using deterministic sequence-family rules over sequence identifiers (e.g., names containing `split_brain`, `stale`, `reorder`, `unknown`, with a control bucket for baseline-good/control families). We report three units of analysis: event-level metrics (TP/FP/FN and latency), sequence-level correctness (`detection_ok`), and stress-profile-level sensitivity (`detection_ok` rate under delay/skew/reorder settings).

6.4 Comparator policies

We evaluate:

- `full_contract`
- `timestamp_only`
- `ownership_only`
- `occ_only` (reference proxy)
- `lease_only` (reference proxy)
- `lock_only` (reference proxy)
- `naive_lww`
- `accept_all`

Comparator caveat. OCC/lease/lock entries are benchmark proxies tied to available event fields; they are not full transaction-manager reimplementations. Their value is ingredient isolation at the coordination boundary: they show which semantic ingredients are necessary to deny the targeted failure classes, not how a production transaction manager would be engineered end to end.

Table 2: Core benchmark summary from `eval.json`. Exact match is measured over the full per-event verdict vector for each sequence.

Corpus	Seq.	Exact match	TP	FP	FN	Precision	Recall/F1
Tiered contracts corpus	54	54/54	37	0	0	1.00	1.00 / 1.00

Table 3: Comparator decomposition from `ablation`. Entries report targeted violations denied versus missed under each policy. Policy names follow artifact identifiers.

Policy	Denied	Missed
`full_contract`	37	0
`timestamp_only`	24	13
`ownership_only`	20	17
`occ_only`	24	13
`lease_only`	24	13
`lock_only`	20	17
`naive_lww`	0	37
`accept_all`	0	37

7 Results

We organize results around the following questions: (RQ1) Does the full contract match ground truth on the benchmark? (RQ2) How do weaker comparators fail structurally? (RQ3) How do detection metrics decompose by inferred failure class? (RQ4) What is validation overhead in the evaluated harness? (RQ5) What does transport-boundary parity show? (RQ6) What does the async stress run characterize?

7.1 RQ1: Exact match on the benchmark

Table **??** answers RQ1: the contract matches the expected verdict vector on every sequence.

7.2 RQ2: Structural failure of weaker policies

The structural pattern is stable and substantively important: temporal-only comparators miss split-brain conflicts; authority-only comparators miss stale and reorder-sensitive classes. This is the core empirical reason the paper should not be read as "timestamps plus bookkeeping." The decisive behavior comes from combining authority and time semantics in one admission rule. Compared to `timestamp_only`, the full contract prevents 13 additional missed violations; compared to `ownership_only`, it prevents 17 additional missed violations.

Comparator failure vignette. In split-brain sequences with concurrent claims on the same key, `timestamp_only` and temporal proxies can accept conflicting ownership updates when timestamps are admissible, whereas authority-aware policies deny the second claim. Conversely, stale-write and reorder-sensitive sequences expose the weakness of ownership-only checks when temporal admissibility is omitted.

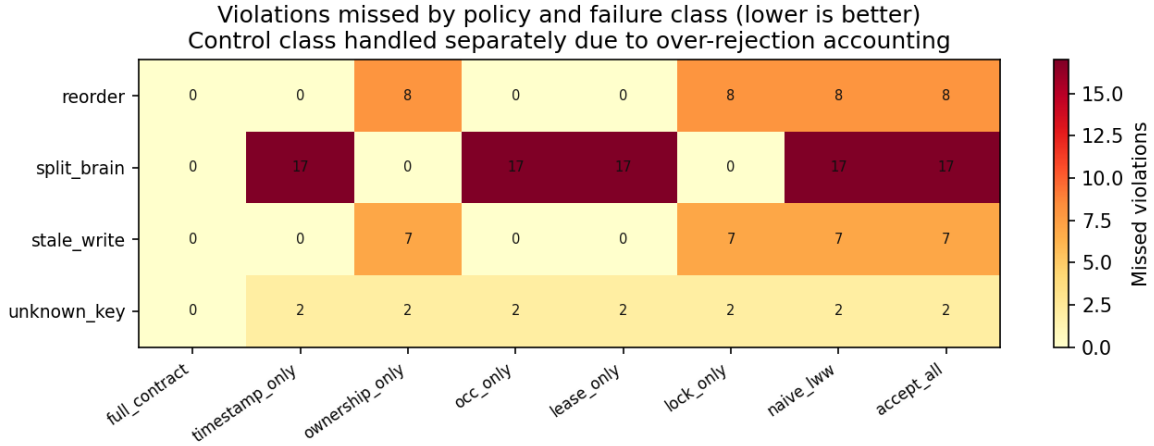

Figure 2: Violations *missed* by policy and inferred failure class (darker is worse). Excludes the `control` class where negative "missed" counts indicate over-rejection in baseline accounting; see `ablation_by_class` in the artifact.

Table 4: Detection metrics by inferred failure class from artifact field `detection_metrics_by_class`.

Class	TP	FP	FN	Precision	Recall	F1
control	3	0	0	1.0	1.0	1.0
reorder	8	0	0	1.0	1.0	1.0
split_brain	17	0	0	1.0	1.0	1.0
stale_write	7	0	0	1.0	1.0	1.0
unknown_key	2	0	0	1.0	1.0	1.0

7.3 RQ3: Class-level diagnostics

The perfect per-class scores should be read together with these interval widths. The benchmark result is strong, but the uncertainty bands remain wider for smaller classes such as `unknown_key` and the control subset. This is exactly why the paper should frame the result as benchmark-backed evidence rather than as an unrestricted field-performance guarantee. For `unknown_key`, the wide interval reflects a small sample regime rather than weak observed performance; this class should be interpreted with caution.

7.4 RQ4: Overhead and scale

Event-level latency is bounded in this evaluated setting: median 4.6 μs, p95 11.8 μs, p99 84.5 μs, with bootstrap confidence intervals of [4.1, 5.4], [9.2, 27.2], and [19.1, 141.9] μs respectively.

7.5 RQ5: Transport-boundary parity

All canonical parity checks pass (`parity_ok_all=true`; parity rate 1.0 over 6/6 checked events). This supports boundary semantic consistency only; it is not a claim of end-to-end transport equivalence across full runtime stacks.

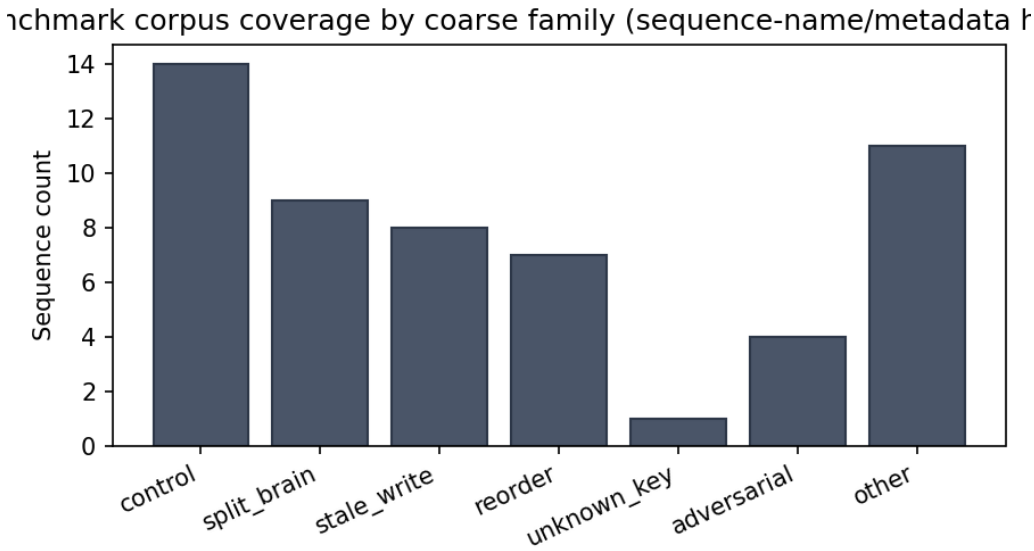

Figure 3: Corpus coverage by coarse family (heuristic from sequence name prefixes). Supplementary diversity is described in the corpus and appendix.

Table 5: Per-class detection-rate 95% confidence intervals from artifact field `excellence_metrics.per_class_ci95`.

Class	Rate	95% CI lower	95% CI upper
control	1.0	0.4385	1.0
reorder	1.0	0.6756	1.0
split_brain	1.0	0.8157	1.0
stale_write	1.0	0.6457	1.0
unknown_key	1.0	0.3424	1.0

7.6 RQ6: Async stress characterization

The artifact includes an async stress runner that sweeps delay, clock-skew, and reorder profiles. We treat this as robustness characterization, not as a substitute for the primary corpus evaluation. Figure **??** summarizes detection retention by profile when `stress_results.json` is present; the main correctness claim remains anchored in RQ1. Sensitivity is dominated by profile-induced reordering and timestamp perturbation rather than by pure delay in isolation.

8 Discussion

The key claim is not that transport can be ignored; it is that transport-level correctness and coordination-level admissibility are distinct. Coordination Contracts isolate that missing layer with explicit predicates and replayable evidence. This provides operational value in post-incident reconstruction, reproducible debugging, and explicit denial reasoning.

This replayability matters especially in regulated or safety-relevant workflows. If a deny is tied to explicit predicates and reason codes, an auditor can reconstruct why a write was rejected, distinguish

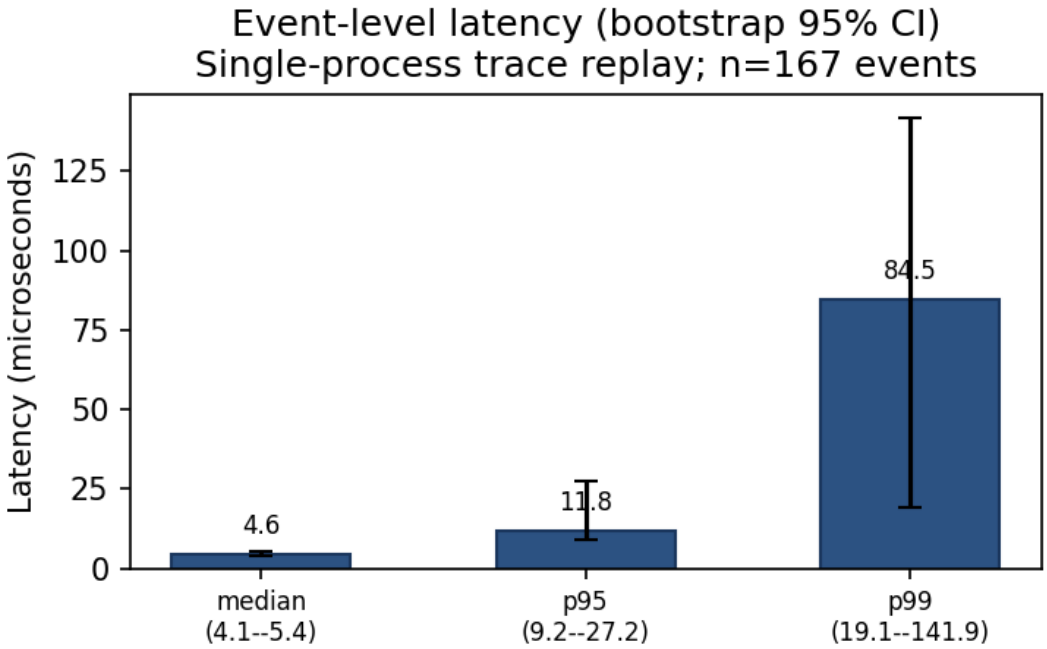

Figure 4: Event-level latency percentiles with bootstrap 95% CIs (single-process trace replay; `latency_percentiles_us` in `eval.json`).

Table 6: Boundary parity on canonical sequences from `transport_parity.json`. 'yes' denotes identical verdict vectors across the compared ingestion paths.

Sequence	Event count	Parity
`good_sequence`	2	yes
`split_brain_sequence`	1	yes
`reorder_sequence`	2	yes
`actor_payload_writer_mismatch`	1	yes

transport success from coordination invalidity, and trace the decision back to the admitted-prefix state without privileged runtime metadata.

9 Limitations and threats to validity

- **Synthetic benchmark scope:** corpus is deliberate but synthetic.
- **Corpus authorship:** sequences and expected verdicts are produced within this project; mitigate by frozen fingerprints, independent re-runs, and benchmark documentation of labeling rules.
- **Single-process evaluation:** no live distributed deployment claims.
- **Identity trust in trace fields:** Byzantine identity spoofing out of scope.
- **Boundary-level transport claim:** parity is boundary semantics, not live-system equivalence.

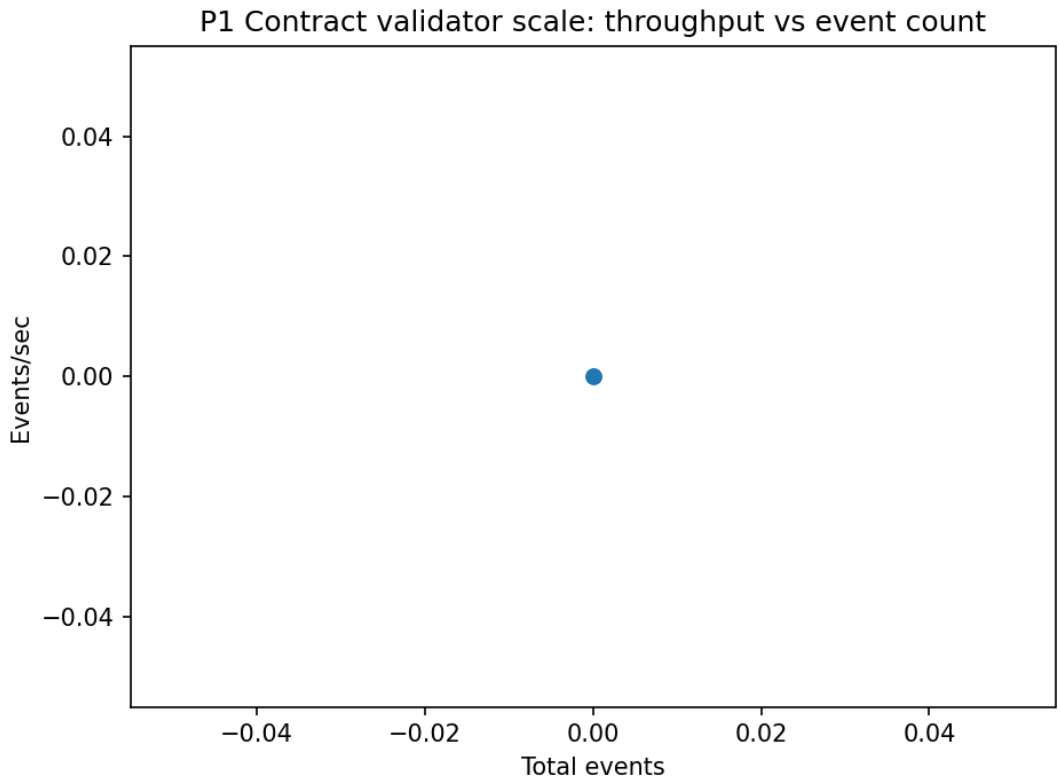

Figure 5: Scale-throughput curve from the current P1 scale-test artifact. This figure is interpretive only: it characterizes the reference implementation in the evaluated single-process setting and should not be read as a deployment-wide performance guarantee.

- **Comparator scope:** OCC/lease/lock are reference proxies in this corpus model.
- **Stress interpretation scope:** async stress outputs characterize sensitivity under delay/skew/reorder profiles, but they do not substitute for multi-process or networked deployment evidence.
- **Canonical non-goals:** This paper does not claim Byzantine tolerance, partition-safe consensus, or live cross-transport deployment equivalence.

10 Conclusion

Coordination Contracts provide a trace-derived admission layer for shared CPS state: explicit keyed authority, temporal admissibility, and reason-coded denials at the write boundary. On the current benchmark artifact, the approach matches expected verdict vectors across all corpus sequences with bounded per-write overhead in the evaluated setting, and its comparator decomposition shows why weaker policies fail. This paper does not claim Byzantine tolerance, partition-safe consensus, or live cross-transport deployment equivalence.The contribution is strongest when read as a reusable and auditable semantics layer with explicit limits, not as a surrogate for consensus, authentication, or full distributed-runtime verification.

References

[1] Reid G. Smith. The Contract Net Protocol: High-Level Communication and Control in a Distributed Problem Solver. *IEEE Transactions on Computers*, C-29(12):1104–1113, 1980.

[2] Leslie Lamport. *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley, 2002.

[3] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall. Robot Operating System 2: Design, architecture, and uses in the wild. *Science Robotics*, 7(66):eabm6074, 2022.

[4] G. Pardo-Castellote. OMG Data-Distribution Service: Architectural overview. In *Proceedings of the 23rd International Conference on Distributed Computing Systems Workshops*, 2003.

[5] V. Mayoral Vilches, R. White, G. Caiazza, and M. Arguedas. SROS2: Usable cyber security tools for ROS 2. In *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2022.

[6] J. Kreps, N. Narkhede, and J. Rao. Kafka: A distributed messaging system for log processing. In *Proceedings of the NetDB Workshop*, 2011.

[7] Apache Kafka Documentation. Topic configuration: `cleanup.policy`. Accessed March 2026.

[8] H. T. Kung and J. T. Robinson. On optimistic methods for concurrency control. *ACM Transactions on Database Systems*, 6(2):213–226, 1981.

[9] C. Gray and D. Cheriton. Leases: An efficient fault-tolerant mechanism for distributed file cache consistency. In *Proceedings of the Twelfth ACM Symposium on Operating Systems Principles*, pages 202–210, 1989.

[10] OPC Foundation. OPC 30500-1: Laboratory and Analytical Devices. Version 1.00, 2025.

[11] OPC Foundation. Laboratory and Analytical Devices: General information on LADS and OPC UA. Version 1.00, 2025.

A Reproducibility block

From repository root:

1. `PYTHONPATH=impl/src python scripts/contracts_eval.py -out datasets/runs/contracts_eval`
2. `PYTHONPATH=impl/src python scripts/contracts_transport_parity.py -out datasets/runs/contract`
3. `python scripts/export_contracts_corpus_table.py -eval datasets/runs/contracts_eval/eval.jsor`
4. `python scripts/export_p1_appendix_tex.py -eval datasets/runs/contracts_eval/eval.json`
5. `python scripts/export_p1_contract_flow.py`
6. `python scripts/render_p1_flow_figure.py`
7. `python scripts/plot_contracts_scale.py`
8. `python scripts/plot_p1_paper_figures.py -eval datasets/runs/contracts_eval/eval.json`

9. `PYTHONPATH=impl/src python scripts/contracts_async_stress.py -out datasets/runs/contracts_ev` (optional; produces `stress_results.json`)

The run used for this draft should record `run_manifest.script_version`, `run_manifest.corpus_fingerprint` and `run_manifest.corpus_sequence_count` in `eval.json`. One-shot generation: `python scripts/generate_pap -paper P1`.

B Full per-sequence corpus summary

Table 7: Per-sequence corpus summary (frozen run).

Sequence	Detection OK	Contract denials
actor_payload_writer_mismatch	yes	1
burst_duplicate_three	yes	0
concurrent_controller_race	yes	2
conflicting_writes_independent_keys	yes	0
coordination_message_passthrough	yes	0
cross_key_interleaved_race	yes	0
delayed_release_reassignment	yes	1
duplicate_delivery	yes	0
edge_case_timestamps	yes	0
good_eight_events	yes	0
good_five_events	yes	0
good_four_events	yes	0
good_interleaved_6	yes	0
good_sequence	yes	0
good_sequence_4events	yes	0
good_seven_events	yes	0
good_single_allow	yes	0
good_six_keys	yes	0
good_ten_events	yes	0
good_then_unknown_key	yes	1
good_three_events	yes	0
good_three_keys_cycle	yes	0
good_two_keys	yes	0
long_horizon_10	yes	0
mixed_key_contention_8	yes	0
multi_key_no_conflict	yes	0
multi_writer_contention	yes	1
reorder_burst_four	yes	1
reorder_first_deny	yes	1
reorder_first_event_deny	yes	1
reorder_mid_chain	yes	1
reorder_multi_key	yes	1
reorder_sequence	yes	1
reorder_three_events	yes	1

Sequence	Detection OK	Contract denials
same_ts_same_writer	yes	0
split_brain_delayed_conflict	yes	1
split_brain_five_writers	yes	4
split_brain_four_writers	yes	3
split_brain_same_epoch	yes	1
split_brain_second_key	yes	1
split_brain_sequence	yes	1
split_brain_then_allow	yes	1
split_brain_three_writers	yes	2
split_brain_two_keys	yes	1
stale_then_allow	yes	1
stale_write_delayed_delivery	yes	1
stale_write_exact	yes	0
stale_write_margin	yes	1
stale_write_sequence	yes	1
stale_write_sequence_2	yes	1
stale_write_sequence_3	yes	1
stale_write_sequence_4	yes	1
unknown_key_deny	yes	1
unsafe_lww_sequence	yes	1

Run manifest (reproducibility). Script version `v0.3`; corpus fingerprint `ae76bbcc00c1b194`; sequence count 54.

C How to regenerate main-text numbers

- Table **??**, Table **??**, Table **??**, Table **??**: from `datasets/runs/contracts_eval/eval.json`.
- Table **??**: from `datasets/runs/contracts_eval/transport_parity.json`.
- Figures **??**, **??**, **??**, **??**: `python scripts/plot_p1_paper_figures.py -eval <eval.json> [-stress <stress_results.json>]`.
- Figure **??**: `python scripts/render_p1_flow_figure.py`.
- Figure **??**: `python scripts/plot_contracts_scale.py`.

D Artifact schema snippets

`eval.json` (key fields). `run_manifest{script_version, corpus_fingerprint, corpus_sequence_count}`, `sequences[]`, `ablation`, `ablation_by_class`, `detection_metrics_by_class`, `latency_percentiles_us`, `excellence_metrics`.

`transport_parity.json` (key fields). `parity_ok_all`, `per_sequence[]`, `parity_confidence{matching_even`

`stress_results.json` (key fields). `stress_sweep`, `results[]{sequence,stress_profile,detection_ok,la`
`run_manifest{delay_sweep,skew_sweep,reorder_probs,seed}`.

E Proxy comparator limits

- `occ_only`, `lease_only`, and `lock_only` are benchmark proxies over available event fields.
- They do not model full read sets, lease renewal windows, or lock-acquisition protocols.
- Their purpose is boundary ingredient isolation, not production-equivalent transaction manager emulation.

F Threats-to-reproducibility checklist

- Freeze and report `run_manifest` values in cited artifacts.
- Regenerate all figures/tables from the same run directory before submission.
- Record Python and plotting backend versions when exporting figures.
- Ensure no placeholder figure/table remains in the compiled manuscript.

G Reviewer-objection map

- **"Baselines are weak."** Addressed by explicit proxy caveat and class-level comparator decomposition.
- **"Synthetic benchmark overfits."** Addressed by anti-circular labeling governance, frozen manifests, and per-sequence appendix reporting.
- **"Transport claim is too broad."** Addressed by boundary-only parity wording and explicit non-goals.
- **"Where is full formal proof?"** Addressed by Lean-aligned determinism scope and explicit proof-sketch boundaries.

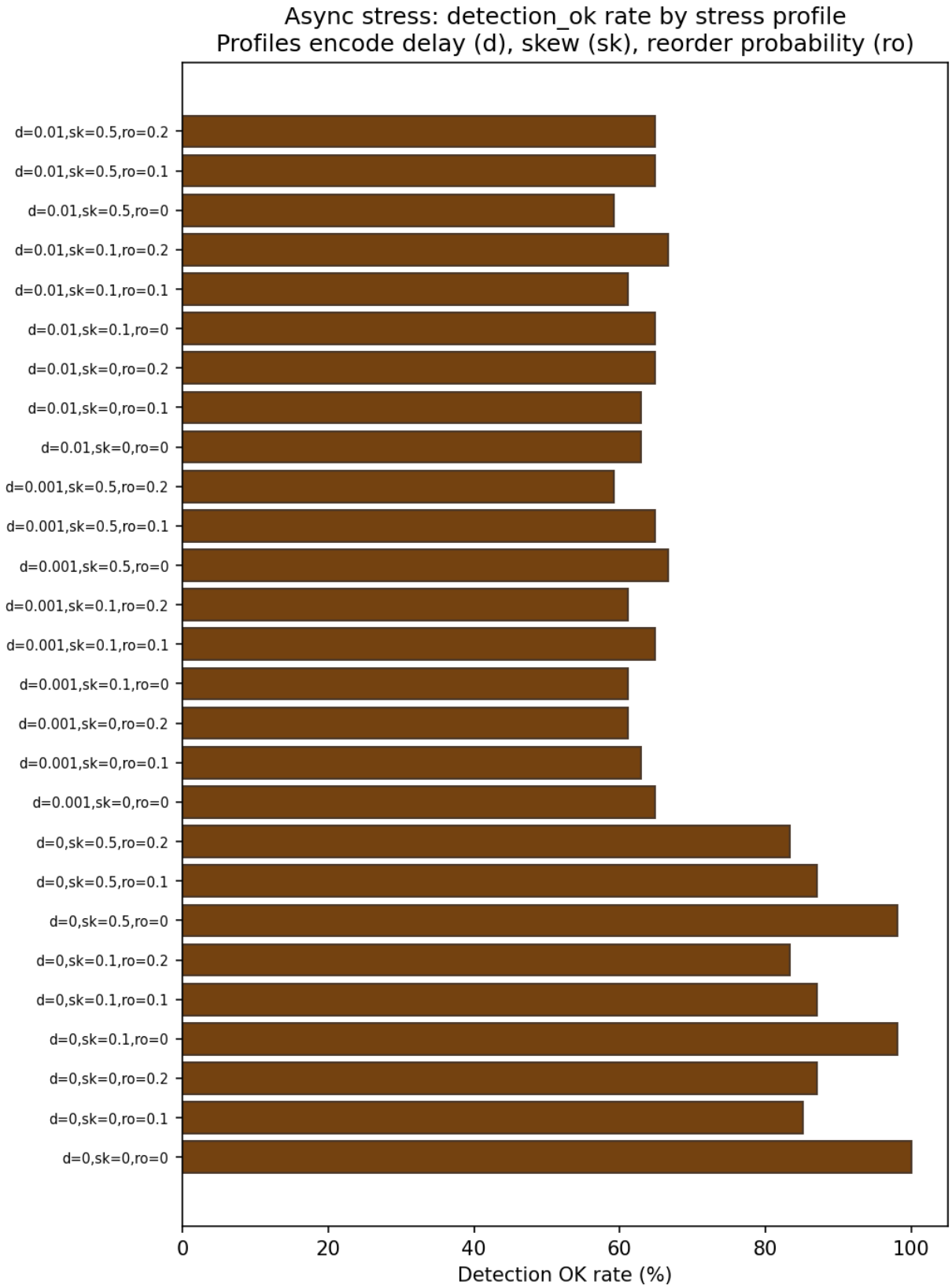

Figure 6: Async stress: fraction of runs with `detection_ok` true, by stress profile (see `stress_results.json`). Interpret as sensitivity analysis, not field deployment.